Experiment 4

Develop a blockchain-powered web application using Solidity and a Metamask-style wallet workflow

Aim

To develop a blockchain-powered application using the Solidity programming language, in place of Remix IDE embedded with a Metamask wallet, and deploy a Crowdfunding smart contract mini-project: a campaign owner sets a funding goal and deadline, contributors pledge ETH, the owner can withdraw only if the goal is reached, and contributors can reclaim their funds if it is not.

Theory

A Metamask-connected Remix IDE session lets a developer compile a Solidity contract in the browser and push transactions through an injected wallet provider to whichever network Metamask is pointed at (a public testnet such as Sepolia, or a local Ganache instance). Hardhat's standalone node (`npx hardhat node`) provides the same transaction-signing, gas-accounting, block-mining behaviour without a browser: it starts a JSON-RPC endpoint on `http://127.0.0.1:8545` pre-funded with 20 deterministic test accounts, and a Node.js script using `ethers.js` (bundled by `@nomicfoundation/hardhat-toolbox`) plays the role Metamask would — holding a signer's private key and sending signed transactions to that endpoint — while `npx hardhat run <script> --network localhost` is the scripted equivalent of clicking "Deploy" in Remix with Metamask's injected provider active.

The mini-project contract, `Crowdfunding.sol`, models the funding lifecycle of a typical crowdfunding dApp:

- The **constructor** takes a `goal` (in wei) and a `durationSeconds`, and records `deadline = block.timestamp + durationSeconds`.
- **`contribute()`** is `payable`; anyone can call it before the deadline, and each call adds `msg.value` to both a per-address `contributions` mapping and the running `totalRaised`.

- **`withdraw()`**, guarded by an `onlyOwner` modifier, only succeeds once the deadline has passed *and* `totalRaised >= goal`; it forwards the full contract balance to the owner and sets a `withdrawn` flag so it cannot be called twice.
- **`refund()`** is the mirror case: once the deadline has passed *without* the goal being met, each contributor can reclaim exactly what they put in; a contributor's balance is zeroed before the ETH transfer (checks-effects-interactions) to prevent re-entrancy, and a second call for the same address reverts with "No contribution to refund".

Because the deadline check depends on `block.timestamp`, the local Hardhat node's `evm_increaseTime` + `evm_mine` RPC methods are used to fast-forward the chain past the campaign deadline within the script — something not possible against a real testnet, which is one of the practical advantages of developing against a local node before deploying to Sepolia/RSK through Metamask.

Tools used: Node.js 24, Hardhat 2.29 with `@nomicfoundation/hardhat-toolbox` (bundles `ethers.js` v6), Solidity 0.8.24. Run directly in a terminal against a standalone `npx hardhat node`, in place of Remix IDE + Metamask, since this was done without a browser wallet.

Output

1. Compile the Crowdfunding contract

```
$ npx hardhat compile
Downloading compiler 0.8.24
Compiled 1 Solidity file successfully (evm target: paris).
```

2. Start a local Hardhat blockchain node

```
$ npx hardhat node
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
========
Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 (10000 ETH)
Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
... (20 accounts total, 10000 ETH each)
```

The local blockchain running as a persistent standalone node, playing the role a Metamask-connected Sepolia/RSK testnet would in the original Remix IDE workflow.

3. Scenario A — a campaign that reaches its goal, owner withdraws

```
$ npx hardhat run scripts/deploy_and_interact.js --network localhost

=== Scenario A: Campaign that reaches its goal ===
Campaign A deployed at: 0x5FbDB2315678afecb367f032d93F642f64180aa3
Goal: 3.0 ETH, duration: 60s
0x70997970C51812dc3A010C7d01b50e0d17dc79C8 contributed 1.0 ETH
0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC contributed 1.0 ETH
0x90F79bf6EB2c4f870365E785982E1f101E93b906 contributed 1.5 ETH
Total raised: 3.5 ETH
Goal reached before deadline? true
Fast-forwarding chain past the deadline (+65s)...
Owner withdraw tx hash: 0x1601cc01c03a68a95b6657440415787ab4c2f6187581cb23c66ec5bb774aa3b5
Owner balance change: +3.5 ETH (gross, before gas)
Contract balance after withdraw: 0.0 ETH
Negative test: owner tries to withdraw a second time...
Reverted as expected: Error: VM Exception while processing transaction: reverted with reason string 'Already
withdrawn'
```

Three contributors (Alice, Bob, Carol) pledge a combined 3.5 ETH against a 3 ETH goal; after the chain is fast-forwarded past the deadline, the owner's `withdraw()` succeeds and drains the full contract balance, and a second withdraw attempt correctly reverts.

4. Scenario B — a campaign that misses its goal, contributor is refunded

```
=== Scenario B: Campaign that misses its goal ===
Campaign B deployed at: 0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9
Goal: 10.0 ETH, duration: 60s
0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65 contributed 2 ETH
Total raised: 2.0 ETH (goal 10.0 ETH)
Fast-forwarding chain past the deadline (+65s)...
Negative test: owner tries to withdraw an unmet goal...
Reverted as expected: Error: VM Exception while processing transaction: reverted with reason string 'Funding
goal not reached'
Dave refund tx hash: 0x5b464219305e435ef54d43b3f3c65b59f6cdce1433e8b774427192de27dc5ac7
Dave balance change: +2.0 ETH (gross, before gas)
Contract balance after refund: 0.0 ETH
Negative test: Dave tries to refund a second time...
Reverted as expected: Error: VM Exception while processing transaction: reverted with reason string 'No
contribution to refund'
```

A second campaign with a 10 ETH goal receives only 2 ETH from Dave; the owner's withdraw attempt correctly reverts since the goal was not met, Dave's `refund()` correctly returns his full 2 ETH, and a repeat refund attempt correctly reverts.

(Full console log saved as `deploy_output.log`, and the Hardhat node's own server-side log as `hardhat_node.log`, both in this folder.)

Conclusion

This experiment developed and deployed a Crowdfunding smart contract — a self-contained blockchain-powered application — in place of the Remix IDE + Metamask workflow, using a scriptable local Hardhat node instead of a browser wallet connected to a public testnet. Two full campaign lifecycles were exercised end-to-end: a goal-reached campaign where three contributors' combined pledges exceeded the target and the owner successfully withdrew the raised funds exactly once, and a goal-missed campaign where the sole contributor correctly reclaimed their full pledge via `refund()` after the deadline passed. Every guard condition in the contract — `onlyOwner`, the deadline check, the goal-reached check, and the double-withdraw/double-refund protections — was independently confirmed by a negative test that reverted with the expected reason string, demonstrating that the same fund-safety guarantees a Metamask-connected Remix deployment would provide hold on this local chain as well.